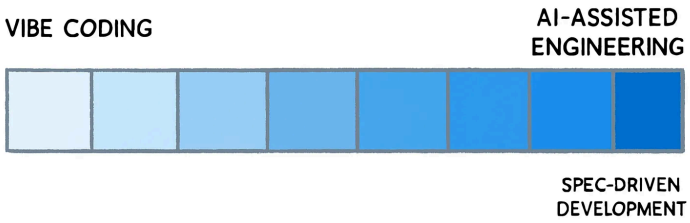
Vibe coding en Ciencia de Datos

El **código asistido por IA** es un espectro.

El "vibe coding" es un enfoque de desarrollo basado en prompts de alto nivel: uno da instrucciones generales, acepta las sugerencias de la IA y se centra en la "vibe" (sensación) general del proyecto en lugar de los detalles de implementación. Este es un enfoque específico en el que normalmente no se lee el código de la IA antes de ejecutarlo.

El vibe coding nunca pretendió describir toda la programación asistida por IA. Hay mucho más que considerar.

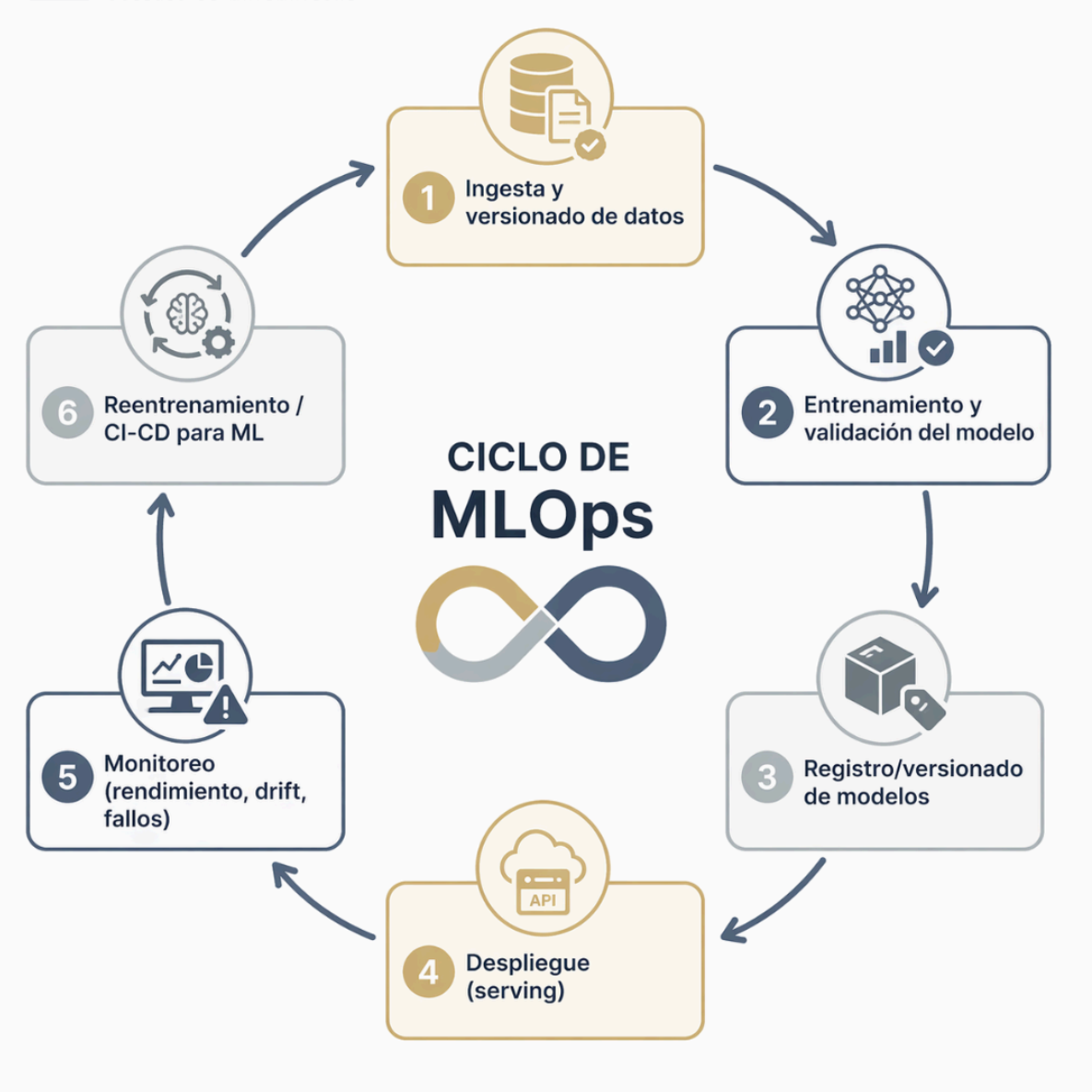


MLOps

MLOps (Machine Learning Operations) es la disciplina que combina ML + DevOps para desarrollar, desplegar, monitorear y mantener modelos de ML en producción de forma confiable y reproducible.

1. Ingesta y versionado de datos
   [AI data-profiling, AI schema inference, AI data-cleaning, AI anomaly detection, AI data labeling/weak supervision]
2. Entrenamiento y validación del modelo
   [AutoML, AI hyperparameter tuning, AI experiment design, AI code-completion, AI eval-set generation]

3. Registro/versionado de modelos
   [AI metadata tagging, AI model card generation, AI experiment summarization]
4. Despliegue (serving)
   [AI infra-as-code generation, AI rollout optimization]
5. Monitoreo (rendimiento, drift, fallos)
   [AI anomaly detection, AI drift detection, AI auto-triage, AI root-cause analysis, AI alert summarization]
6. Reentrenamiento / CI-CD para ML
   [AI pipeline generation, AI trigger optimization, AI test generation, AI continuous evaluation]



Mejores prácticas

1. No programes primero - planifica primero. Usa el Modo Plan, Mejora el Prompt o escribe una SPEC o mini-PRD.
    - Mal prompt: "Constrúyeme una app de tareas"
    - Buen prompt: "Dame algunas opciones para la arquitectura de una app de tareas, empezando por la más sencilla primero. Todavía no escribas código;

solo describe el enfoque y pregúntame qué dirección tomar."

    - Excelente: Un mini-PRD (Documento de Requisitos del Producto) o SPEC.md. Define el problema, describe las bases de datos y especifica los resultados deseados.

2. Proporciona la documentación adecuada.
    - Revisa el límite de conocimiento del modelo
    - Incluye documentación relevante
    - Establece reglas globales
    - Configura el system prompt. Por ejemplo:

```
Sigue siempre estas directrices:
1. Define el modelo de datos antes de escribir código
2. Comienza con datos ficticios (mock data) en lugar de una base d
3. Crea una librería de componentes y divide el código en múltiple
4. Centraliza la gestión del estado
5. Implementa en pequeños bloques (chunks)
6. Verifica dos veces que estás cambiando los archivos correctos
7. Haz preguntas de seguimiento si los requisitos no están claros
```

3. Crea instrucciones antes debuggear. Indica explícitamente qué intentas lograr. Esta especificidad ayuda a la IA a entender tu intención, no solo el síntoma.
    - Mal prompt: "El botón no funciona"
    - Buen prompt: "Quiero que el botón Guardar almacene los datos del formulario y luego redirija a la página del dashboard. Actualmente no está guardando y muestra este error: [pega el error]"

Prompt engineering

Principios:

- Proporciona suficiente contexto: si entra basura, sale basura; incluye notas sobre arquitectura, librerías, fragmentos de código y mensajes de error exactos.
- Sé específico sobre tu objetivo
- Divide tareas complejas: divide problemas grandes y de varios pasos en bloques pequeños e iterativos para obtener respuestas más enfocadas y manejables.
- Guía a la IA a través de un razonamiento paso a paso para problemas complejos (Chain-of-Thought prompting)
- Incluye ejemplos de entradas y salidas (Few-Shot prompting)

- Aprovecha roles y personas: pide a la IA que "actúe como" una persona específica para influir en el tono, el estilo y la profundidad de las respuestas.
- Prompt basado en restricciones: establece límites y requisitos claros para guiar el resultado de la IA (stack tecnológico, estilo de código, rendimiento).
- Itera y refina tus prompts: revisa las respuestas y proporciona retroalimentación correctiva o preguntas de seguimiento.

Context Engineering

Principios:

- Ensamblaje dinámico de información: comparte archivos, carpetas o fragmentos de código específicos que sean centrales para la solicitud.
- Degradación de contexto (Context Rot): resume y refresca el contexto periódicamente.
- Contexto de código relevante: comparte archivos, carpetas o fragmentos de código específicos que sean centrales para la solicitud.
- Piensa como un Sistema Operativo: considera el modelo de IA como una CPU y su ventana de contexto como la RAM. Carga la información necesaria en la memoria de trabajo para cada tarea específica.
- Documentos de diseño: incluye secciones relevantes de los documentos de diseño para proporcionar un contexto más amplio del proyecto.

Avanzados:

- RAG (Generación Aumentada por Recuperación)
- Herramientas (Tooling)

Principios de producción

- Revisa siempre el código generado por IA
- Ten una estrategia de pruebas exhaustiva: la IA puede ayudar a generar pruebas, pero tú debes verificar la cobertura y la calidad.
- Desarrollo centrado en la seguridad